Claude Code on Raspberry Pi: Use and Best Practices

Oct 27, 2026 · Joe Solch

What you get

One Raspberry Pi per person, with Claude Code running as one normal user and never as root. There are two deploy scripts: `Claude_Code_Raspberry_Pi_5_Deployer.sh` for a Pi 5 (4 GB or 8 GB) and `Claude_Code_Raspberry_Pi_4_Deployer.sh` for a Pi 4 Model B (8 GB). Everything is installed straight onto the OS: no Docker, no LXC and no other containers. Claude Desktop runs on the Pi's own desktop with Chat and Code. Cowork is left out on purpose, because it needs its own virtual machine, which a Pi can't spare.

Part	Where it lives	Owned by
Claude Code	~/.local/bin/claude	the user
Claude settings, plugins, skills	~/.claude/	the user
Projects	/project	the user
Claude Desktop (Chat, Code)	`claude-desktop`, from Anthropic's apt repository (arm64)	root; runs as the user on the desktop
Node.js, Python, Go, language servers	/usr, /usr/local	root
Rust toolchain	/opt/rust (your crates in ~/.cargo)	root
GitHub CLI (`gh`)	/usr/bin/gh, from GitHub's apt repository	root; you sign in as the user
Playwright browsers	/opt/ms-playwright	root; everyone can read them
Web UI (CloudCLI)	port 3001, service `cloudcli`; optional on the Pi 5	runs as the user
Update and health tools	`agentic-update`, `agentic-doctor`, `agentic-setup-user`, `agentic-errorlog`	root (run with sudo)
Problem log	Agentic-Errors.txt on the user's desktop, only when something fails	the user
Phone alerts (optional)	ntfy, set in /etc/agentic/ntfy.conf	root
Memory settings (Pi 5, 4gb profile)	/etc/agentic/memory.env, /etc/sysctl.d/90-agentic-memory.conf, /etc/rpi/swap.conf.d/90-agentic-memory.conf	root
Memory settings (Pi 4)	/etc/sysctl.d/90-agentic-pi4.conf	root

Part	Where it lives	Owned by
Notes card	/etc/agentic/README.md (copy in ~/CLAUDE-PI.md)	root

The comment block at the top of each script has a "Where things live" list of every file it writes, and comments through the script explain each step.

Before you run the script

1. **Pick the OS.** Use a 64-bit image with a desktop: Raspberry Pi OS (Debian 12 Bookworm or 13 Trixie) or Ubuntu 24.04 / 26.04 Desktop for Raspberry Pi. Ubuntu is officially supported by Claude Desktop; Raspberry Pi OS works but Anthropic hasn't tested it. A Lite or Server image works for everything except Claude Desktop.
2. **Boot from an SSD if you can.** On a Pi 5 use an NVMe drive; on a Pi 4 use a USB 3 SSD in a blue port. An SD card makes apt, cargo and npm painfully slow.
3. **Power and cooling.** Use the official supply (27 W USB-C for a Pi 5, 15 W for a Pi 4) and a cooler or fan case. Builds keep every core busy for minutes, and a weak supply or a hot chip makes the Pi slow itself down. `agentic-doctor` warns when that happens.
4. **Create your own account** in Raspberry Pi Imager or the first-boot wizard. That account is the natural choice for the Claude user, because it's the one the desktop logs in as.
5. **Run the script** from a terminal on the Pi: `sudo bash Claude_Code_Raspberry_Pi_5_Deployer.sh` (or the Pi 4 one). It asks all its questions first and changes nothing until you answer "Proceed? y".

The script asks for the hostname, timezone, root password, the Claude user, optional ntfy alerts, whether to install Claude Desktop, and whether the web UI is reachable from the Pi only (the default) or from your LAN. The Pi 5 script also asks for the memory profile, the web UI and the 4K-page kernel; the Pi 4 script asks about its boot tuning instead. Both are safe to run again: existing users, settings and shell blocks are kept and installed parts are skipped.

Pi 5 memory profiles (4gb and 8gb)

The Pi 5 script has one setting for each board size. It suggests the profile from the RAM it finds, and you can override it.

	4gb (lean)	8gb (standard)
zram (compressed swap in RAM)	Sized to 100% of RAM; zstd packs about 3:1, so 4 GB of swap costs well under 2 GB of RAM	50% of RAM, or the OS's own zram as it is
Kernel memory settings	swappiness 150, page-cluster 0, tighter dirty-page limits	Kernel defaults
Parallel builds	2 jobs for cargo, make, go and npm	All cores
Node.js heap	Capped at 1.5 GB	Node's default

	4gb (lean)	8gb (standard)
Web UI (CloudCLI)	Off by default (uses about 200 MB all the time)	On by default

The build-job and heap limits live in `/etc/agentic/memory.env`. They're applied to login shells, to `/etc/environment` (so Claude Desktop sees them too) and to the web UI's service.

zram on Raspberry Pi OS Trixie. Trixie ships its own zram service, `rpi-swap`, which caps zram at 2 GB. On the 4gb profile the script adds a small setting file that raises it to the full RAM size, but `rpi-swap` only reads it at boot, so **reboot once after the deploy**. Until then the summary, the notes card and `agentic-doctor` all show the real size ("zram now: ..."), and the doctor warns while zram is below 75% of RAM. On other systems the script sets zram up itself with `zram-tools`.

Changing profile. Run the script again and pick the other profile. Going from 4gb to 8gb removes the 4gb settings. On a re-run the script also notices an installed web UI and asks whether to keep it; the default follows the profile, so on 4gb pressing Enter removes it. Removing it stops and deletes the service and uninstalls the package, but keeps its login database (~/.cloudcli), so turning it back on later keeps your account.

Pi 4. The Pi 4 script has no profiles. It targets 8 GB, uses zram at 50% of RAM (or the OS's own), swappiness 100 and 2 cargo jobs (set in /etc/environment, so Claude Desktop and the web UI get it too), because the Pi 4's slower cores gain little from more. The web UI is always installed.

First steps after deploy

Do these right after the script finishes, in this order.

1. **Reboot.** The script offers a reboot at the end. It's needed for the 4K-page kernel (Pi 5), the larger zram (Pi 5, 4gb), the Pi 4 boot tuning, and kernel updates.
2. **Claim the web UI, if installed.** It listens only on the Pi unless you chose LAN at deploy. From your PC, run `ssh -L 3001:localhost:3001 <user>@<ip>` and open `http://localhost:3001` (with LAN access, open `http://<ip>:3001` instead). Create its login straight away: until you do, anyone who can reach it can create it.
3. **Sign in to Claude Desktop.** Log in to the desktop as your user, open **Claude** from the menu (or run `claude-desktop`) and sign in. Chat and Code work; Computer Use and dictation aren't available on Linux yet.
4. **Sign in to Claude Code.** In a terminal, run `claude` and follow `/login`. A claude.ai subscription also enables Remote Control.
5. **Check health.** Run `sudo agentic-doctor`. The expected WARN is "logged in", and only before step 4. Any WARN or FAIL is also copied to Agentic-Errors.txt on your desktop, so if you ran it early, delete that file.
6. **Set your git identity.** Run `git config --global user.name "..."` and `git config --global user.email "..."`.
7. **Sign in to GitHub.** Run `gh auth login` once (choose HTTPS and let it set up git credentials).

8. **Keep the install log.** It's under `/var/log/agentic-deploy/`, with the provisioning trace in `/var/log/agentic-provision.log`.

The 4K-page kernel (Pi 5)

Raspberry Pi OS boots the Pi 5 with a 16K-page kernel by default. Some arm64 programs built with jemalloc crash on it, including the ripgrep that comes bundled with Claude Code. The script offers to switch to the standard 4K-page kernel (it adds `kernel=kernel8.img` under `[pi5]` in /boot/firmware/config.txt), which takes effect after a reboot. Either way, Claude Code is set to use the system's ripgrep (`USE_BUILTIN_RIPGREP=0`), so search works on both. The Pi 4 kernel already uses 4K pages.

Daily use

Log in as your user and run `claude`; the shell starts in /project. Claude reads that project's files and the shared /project/CLAUDE.md.

- **Start a project:** `mkdir -p /project/src/myapp && cd $_ && git init`, then `claude`.
- **Resume:** `claude --continue` picks up the last session in that folder; `claude --resume` lets you choose one.
- **Claude Desktop:** the Code tab works on /project directly.
- **Web UI:** chat, a file editor, git and a shell, all running as your user. By default it's reachable only through an SSH tunnel: `ssh -L 3001:localhost:3001 <user>@<ip>`, then `http://localhost:3001`. Re-run the deploy script and answer `lan` to open it on `http://<ip>:3001`. To open it to your LAN later, change `HOST=127.0.0.1` to `HOST=0.0.0.0` in /etc/systemd/system/cloudcli.service, then run `sudo systemctl daemon-reload && sudo systemctl restart cloudcli`.
- **From another PC:** Claude Desktop on your own computer can open an SSH session to the Pi (`ssh-copy-id <user>@<ip>` first; Desktop can't type passwords). `ssh <user>@<hostname>.local` also works on most networks.
- **From your phone:** in a session, type `/rc` (Remote Control), then continue from the Claude app or claude.ai/code.
- **No Docker:** there are no containers here. If a project needs a database server, run `sudo apt install postgresql` (or `redis-server`) yourself and use it as a normal service.
- **Admin commands:** run `sudo apt install ...` and other root tasks yourself. Claude is blocked from sudo and su, so when it needs one it should ask you.
- **Python:** use a venv per project (`python3 -m venv .venv`) rather than system-wide installs.
- **Node:** `npm i -g` installs into your own ~/.local, so no sudo is needed. npm skips install scripts for packages you haven't approved and lists them after the install; approve one you trust with `npm approve-scripts` in that project, or `--allow-scripts=<pkg>` on a global install.

Maintenance

Updates run on their own every Sunday at 4 AM local time; you only step in when the health check fails.

`agentic-update` does one pass in six steps: OS packages (including Claude Desktop and the Pi's kernel and firmware), npm and the global dev tools (including Playwright and its Chromium, and gopls), Rust, Claude Code, the web UI (if installed), then a health check.

When an update or health check hits a WARN or FAIL, an entry is appended to `Agentic-Errors.txt` on your desktop (your home folder if there is no desktop). Each entry has the time, the step that failed and the last 40 lines of the log. If the file is there, something needs a look; if it isn't, every run since you last deleted it was clean. Entries older than 90 days are trimmed and the file is capped at 256 KB. Delete it once you've dealt with the problems.

Task	Command	Notes
Update everything now	`sudo agentic-update`	The six steps above
Health check	`sudo agentic-doctor`	Exit 0 means healthy; also checks memory, zram, page size, under-voltage and throttling (and temperature on the Pi 4), and warns if sudo stops asking for a password
Check for problems	`cat ~/Desktop/Agentic-Errors.txt`	Only exists after a WARN or FAIL; delete it when dealt with
Test phone alerts	`sudo agentic-errorlog --test-ntfy`	Only if ntfy is set up; see the next section
See the real zram size	`cat /etc/agentic/zram-status`, `swapon --show`	Pi 5
Read the update log	`less /var/log/agentic-update.log`	Rotated monthly
Repair the Claude setup	`sudo agentic-setup-user`	Reinstalls Claude if it's missing or broken, re-adds the plugins and skills, and adds any new deny rules; the rest of your settings.json is kept
Web UI status and logs	`systemctl status cloudcli`, `journalctl -u cloudcli`	If installed
Install logs	`/var/log/agentic-deploy/`, `/var/log/agentic-provision.log`	The provision log includes a trace of every command

A new kernel or firmware from an update only takes effect after a reboot; the update log says when one is needed. Back up /project and ~/.claude regularly; an SD card or SSD can fail without warning.

Phone alerts with ntfy (optional)

Agentic-Errors.txt only helps when you're at the Pi. If you also want a push on your phone, `agentic-errorlog` can send one through ntfy every time it writes an entry. It stays off until you set a topic. The push is short (how many problems, the first few lines, and which Pi); the file stays the full record.

1. **Pick a topic name that's hard to guess.** On the public ntfy.sh server a topic has no password: anyone who knows its name can read your alerts and post fake ones. Use something long and random, such as `claude-alerts-` followed by the output of `openssl rand -hex 8`. Avoid your name, the hostname, or anything obvious like `claude-errors`.
2. **Subscribe on your phone.** Install the ntfy app (Android or iOS), tap **+**, and enter the same topic (and your server, if you use your own).
3. **Give the topic to the Pi.** Answer the ntfy prompt when you run the deploy script (re-running it and pressing Enter keeps the current setting), or set it later with `sudo nano /etc/agentic/ntfy.conf`.
4. **Test it.** Run `sudo agentic-errorlog --test-ntfy`. A test push should arrive within seconds.

The settings live in `/etc/agentic/ntfy.conf`, which only root can read because it can hold a token:

Setting	Meaning
`NTFY_TOPIC`	Topic name, or a full URL such as `https://ntfy.example.com/my-topic`. Empty turns alerts off.
`NTFY_SERVER`	Server for a bare topic name; default `https://ntfy.sh`. Change it if you run your own ntfy server.
`NTFY_TOKEN`	Access token (`tk_...`) for a server or topic that requires a login; leave empty otherwise.

For more privacy, reserve the topic with an ntfy.sh account or run your own server, and use an access token. To stop alerts, empty `NTFY_TOPIC`.

Best practices with Claude

Treat Claude like a fast, capable colleague with shell access. Give it clear goals and a way to check its work, and review what it changes.

- **Use git for everything.** Commit before a big task, so a bad change is one `git reset` away. Read diffs before you push.
- **Plan first on big tasks.** Press Shift+Tab for plan mode, or use `/superpowers:brainstorm` and then `/superpowers:write-plan`.
- **Give it a way to verify.** Tests, a build command, or the webapp-testing skill for UIs let Claude check itself.
- **Keep secrets out of reach.** Put credentials in `.env` files or a `secrets/` folder; the deny rules stop Claude reading those, `*.pem`, everything in ~/.ssh, and the gh, git and npm credential files. Never paste a token into the chat.
- **Write down project rules.** Add a CLAUDE.md to each project (build, test and style commands); keep the shared /project/CLAUDE.md short.

- **Watch auto mode.** It approves most actions on its own, behind a safety classifier and the deny rules. For risky work, switch to default mode with Shift+Tab, or use `/permissions`.
- **Don't run Claude as root.** The `claude` command refuses to run as root here; keep it that way.
- **Mind the memory.** On a 4 GB Pi run one agent at a time and close Claude Desktop during big builds. On 8 GB, one or two agents at a time, and `cargo build -j2` for big Rust builds.
- **Manage context.** Use `/clear` between unrelated tasks, `/compact` on long ones, and `/effort` to spend more or less thinking.
- **Review plugin and MCP additions.** Anything you install runs as your user; add only sources you trust.

Security model

There's no container around Claude here, so the user account is the boundary. Claude can do whatever your user can, and nothing that needs root without you typing the sudo password.

Layer	What the script does	Why
Claude user	Normal account; root is blocked from running `claude`	Limits what an agent mistake touches
Passwords	The user's password must differ from root's	A leaked user password doesn't become root
sudo	Optional, and asks for a password. Raspberry Pi OS gives the first user passwordless sudo; the script checks every sudoers file for a NOPASSWD rule that covers the user (directly, through ALL, or through a group such as `sudo`) and offers to remove it, checking each edit with `visudo`. `agentic-doctor` warns if one comes back	Admin work stays with you; Claude can't type the password
Claude permissions	Auto mode, plus deny rules for secrets and credential files (~/.ssh, gh, git, npm), `dd`, `mkfs`, `rm -rf /`, `sudo` and `su`	Claude can't read credentials or escalate on its own
npm install scripts	Allowed only for the web UI's four native modules; the weekly tool updates run with none	A compromised package can't run code as root when it's installed
SSH	Root and the user log in with a password or key; at most 4 tries per connection	Convenient; keep port 22 on your LAN
Web UI	Runs as the user; own login; optional on the Pi 5; listens on the Pi only unless you chose LAN at deploy, and starts only after setup is finished	Plain HTTP on port 3001: use an SSH tunnel, or keep LAN access to a trusted network or behind a TLS reverse proxy
Phone alerts	Off unless you set an ntfy topic; settings file is root-only	A guessable topic would let others read your alerts

Layer	What the script does	Why
Updates	Weekly, with health check	Keeps the OS, Pi firmware, Claude Desktop and Claude Code patched

Further hardening you can add:

- Turn on the firewall (`sudo apt install ufw`, `sudo ufw allow from <your-lan> to any port 22 proto tcp`, plus port 3001 if you chose LAN access for the web UI, then `sudo ufw enable`).
- Use SSH keys, and set PasswordAuthentication no in /etc/ssh/sshd_config.d/00-agentic.conf once they work.
- If you never need root over SSH, set PermitRootLogin no in that same file.

Troubleshooting

Start with Agentic-Errors.txt on your desktop if it exists, then `sudo agentic-doctor`; each FAIL line names the part to look at.

Symptom	Check or fix
Deploy failed	Read `/var/log/agentic-provision.log`; the line before "provisioning FAILED" shows the failing command. Fix it and re-run the script; it's safe to run again.
Deploy pauses at "Waiting for … first-boot apt jobs"	A freshly flashed Pi runs its own updates after first boot. The script waits while they hold apt's locks (it prints which process) and carries on after 3 minutes at most; let it run.
Doctor: "4gb: zram at least 75% of RAM" WARN	Reboot. `rpi-swap` only applies the new size at boot. Check with `swapon --show`.
Doctor: "4K-page kernel" WARN (Pi 5)	Reboot after choosing the 4K kernel, or re-run the script and say yes to it.
Doctor: under-voltage or throttling	Use the official power supply and a cooler. `vcgencmd get_throttled` should print `0x0`.
Builds or the desktop freeze	Out of memory. Run fewer agents, close Claude Desktop during builds, or use the 4gb profile. `free -h` shows what's in use.
Claude Desktop forgets your sign-in	The desktop keyring is locked or missing: `sudo apt install gnome-keyring libpam-gnome-keyring`, then log out and in.
Claude Desktop won't start	It needs a desktop session (not a Lite image) and a 64-bit OS. Run `claude-desktop` in a terminal to see the error.
`claude: not run as root`	Expected. Use `su - <user>`, then `claude`.

Symptom	Check or fix
`claude: command not found` as the user	Run `sudo agentic-setup-user`.
Web UI won't open	It's local only unless you chose LAN: use the SSH tunnel above. If you turned it off, re-run the script and say yes. Otherwise check `journalctl -u cloudcli -n 50`, then `sudo agentic-update` (it rebuilds native modules).
Doctor: "sudo asks for a password" WARN	A NOPASSWD rule (or `Defaults !authenticate`) lets the user use sudo without a password, which an auto-mode agent could use. `sudo -l -U <user>` shows it; remove it with `sudo visudo` or by editing the file in /etc/sudoers.d/, or re-run the deploy script and say yes when it offers to.
`sudo` asks for a password Claude can't give	Expected. Run the command yourself.
No ntfy push arrives	Run `sudo agentic-errorlog --test-ntfy`; it names the failure. Check the topic matches the one in the app, and the token if your server needs one.